ActivityGraph Gateway Setup

Turning a Raspberry Pi 4B into the network gateway, so every device on the network is measured rather than just the machine doing the capturing.

A laptop on Wi-Fi can only ever see its own traffic. In managed mode a wireless client receives frames addressed to it plus broadcast, and nothing else, so another device's browsing is invisible no matter how long you capture. That is a property of 802.11, not a limitation of the software.

A gateway is different. Every packet from every device must pass through it to reach the internet. Move the capture there and the blind spot disappears, with no change to the code.

What you are building

```
internet feed
     |
[ Router 1 ]        modem only, Wi-Fi OFF
     |  ethernet
[ Raspberry Pi ]    eth0 = WAN      (built-in port)
     |              eth1 = LAN      (USB adapter)  <-- CAPTURE HERE
     |  ethernet
[ Router 2 ]        access point: DHCP off, static IP
   )  )  )  Wi-Fi
phone / laptops / TV
```

Traffic path: device, router 2, Pi, router 1, internet. Every packet crosses the Pi carrying its original source IP, which is what makes per-device and per-person reporting possible.

> **Read this before you start**
> Three steps in this guide fail *silently*. They produce a dashboard that fills in normally while the data underneath is wrong. They are marked CHECKPOINT. Do not skip the verification at each one.

Before you start

Beyond the two routers and the Pi, you need:

Item	Notes
microSD card, 16 GB+	For Raspberry Pi OS
Pi power supply	USB-C, 3 A
USB Ethernet adapter	Gives the Pi its second network interface
USB-A to USB-C adapter	Only if your Ethernet adapter is USB-C. The Pi's USB-C port is power input only and cannot host peripherals. This is the piece most often missing.
Two Ethernet cables	Router 1 to Pi, Pi to router 2

Time required is roughly 60 to 90 minutes. Your network will be down for a few minutes during Phase 7.

Phase 1: Flash and boot the Pi

On your laptop, open Raspberry Pi Imager and choose **Raspberry Pi OS Lite (64-bit)**. Before writing, open the settings (gear icon) and configure:

- Hostname: `activitygraph`
- Enable SSH, with a password or your public key
- Your username and password
- Your Wi-Fi details are not needed, since the Pi uses Ethernet

Write the card, put it in the Pi, connect the Pi's **built-in** Ethernet port to a LAN port on router 1, and power it on. Wait about a minute, then from your laptop:

```
ssh <your-user>@activitygraph.local
```

If the hostname does not resolve, find the Pi's address in router 1's DHCP client list and use that instead.

Phase 2: The USB adapter [CHECKPOINT]

Plug the Ethernet adapter into a **blue USB 3.0 port**, using the USB-A to USB-C adapter if needed. Black ports are USB 2.0 and cap throughput around 280 Mbps. Then:

```
ip -br link
```

You should see `eth0`, `eth1` and `lo`. Most Realtek and ASIX based adapters need no drivers. If `eth1` is missing, check `dmesg | tail` and confirm the port. **Do not continue until it appears.**

Pin the interface name

Interface names can swap between reboots. Note the adapter's MAC from the output above, then:

```
echo 'SUBSYSTEM=="net", ACTION=="add", ATTR{address}=="AA:BB:CC:DD:EE:FF", NAME="eth1"' \
  | sudo tee /etc/udev/rules.d/10-activitygraph-lan.rules
```

> **Why this matters**
> If eth0 and eth1 swap after a reboot, you capture the WAN side instead of the LAN side. Everything downstream of NAT carries the Pi's own address, so the dashboard would attribute your entire household to a single device. Nothing errors. The numbers just quietly become meaningless.

Phase 3: Get the code onto the Pi

> **Why copy instead of clone**
> The Pi cannot clone a private GitHub repo without credentials, and copying from your laptop avoids setting those up on the Pi. It does bring the laptop's virtual environment along, which is useless on the Pi's architecture and has to be deleted and rebuilt: see the warning further down.

From your **laptop**, in PowerShell. Substitute your own project path and Pi username, and keep the quotes: the path contains a space, and without them scp reads it as two arguments.

```
scp -r "C:\Nathan Daily\Study\Projects\ActivityGraph\activitygraph" nathan@activitygraph.local:~/
```

The destination is `~/` rather than `~/activitygraph`, which makes the result predictable. `scp -r` copies *into* a destination that already exists, and your home directory always exists, so the project lands at `~/activitygraph` every time. That directory is the project root, and every later path in this guide is relative to it. It legitimately contains a folder also named `activitygraph`, which is the Python package: that nested name is normal.

Run the copy once
Running it a second time copies into the project root instead, leaving the real project at `~/activitygraph/activitygraph` and quietly breaking every path below. The transfer sends about 115 MB and takes a few minutes, almost all of it the .venv folder you are about to throw away; scp has no exclude option. To ship a change later see Updating the Pi, below: do not repeat this command, and do not delete the project directory to make room for it.

Updating the Pi later

There is no `git pull` on the Pi. The code got there by scp, so there is no clone to pull into, and Raspberry Pi OS Lite ships without git at all. Copy only what changed, then reinstall the service:

```
scp -r "C:\Nathan Daily\Study\Projects\ActivityGraph\activitygraph\activitygraph" "C:\Nathan Daily\Study\Projects\Ac
```

```
sudo ~/activitygraph/deploy/install-service.sh
```

Never delete the project directory to re-copy
It holds accounts.db, history.db and accounts.key: every account, every recorded day of activity, and the key that keeps people signed in. None of them exists anywhere else. Naming the two subfolders above is what makes an update safe to repeat, because it replaces the Python package and the deployment scripts and touches neither the databases nor .venv. That last one matters more than it looks: the laptop's .venv/pyvenv.cfg says home = C:\Python313, so copying the whole project over an installed Pi overwrites the virtualenv's idea of where Python lives and breaks it in a way that reads as a missing package.

The package is installed with `pip install -e .`, so new code is live as soon as the service restarts. Reinstalling dependencies is only needed if pyproject.toml changed.

Then on the **Pi**, install the system packages:

```
sudo apt update && sudo apt full-upgrade -y
sudo apt install -y python3-pip python3-venv tcpdump dnsmasq nftables
```

rm -rf .venv is required, not optional
The copy brought your laptop's Windows virtualenv, which holds Windows executables in a `Scripts` folder where Linux expects `bin`. Left in place, every `.venv/bin/...` command below fails with a confusing "No such file or directory".

```
cd ~/activitygraph
rm -rf .venv
python3 -m venv .venv
.venv/bin/pip install -e .

.venv/bin/python -m activitygraph.main --demo
```

The demo run needs no root and no network traffic. If it prints the category and time tables, the software side is working. `tcpdump` is installed because it brings in libpcap and gives you a second tool for checking what an interface is actually receiving.

If anything above could not find the project, confirm where it landed. Whatever directory holds `pyproject.toml` is the project root, and should be `/home/<you>/activitygraph`. If it is one level deeper, the copy ran twice: use the path it reports for the rest of this guide, including the systemd unit in Phase 10.

```
find ~ -name pyproject.toml -not -path '*/.venv/*' 2>/dev/null
```

Phase 4: Make the Pi a gateway

4.1 LAN address

Give `eth1` a static address on its own subnet, kept distinct from router 1's `192.168.1.x` so routing is unambiguous. Leave `eth0` alone: it takes a DHCP lease from router 1 as normal.

> **Check which network manager you have first**
> Raspberry Pi OS switched from dhcpcd to NetworkManager in Bookworm (October 2023). `/etc/dhcpcd.conf` is often still present on such a system: it exists, looks entirely editable, and is completely ignored. Its presence is not evidence that dhcpcd is in use.

```
systemctl is-active NetworkManager && ls -l /etc/dhcpcd.conf
```

NetworkManager, which is almost certainly your case on a fresh install:

```
sudo nmcli connection add \
  type ethernet ifname eth1 con-name ag-lan \
  ipv4.method manual ipv4.addresses 192.168.50.1/24 \
  ipv4.never-default yes ipv6.method disabled

sudo nmcli connection up ag-lan
```

- **No gateway is set.** eth1 faces your devices, not the internet. The default route must stay on eth0.
- **ipv4.never-default yes** guarantees eth1 can never become the default route, which protects the capture invariant: an interface carrying the default route is by definition the WAN side.
- **ipv6.method disabled** keeps the LAN single-stack, so devices cannot pick up IPv6 routes that bypass NAT and therefore bypass measurement.

ActivityGraph does capture IPv6 where it is present, so that last setting is belt and braces rather than the only defence. It still matters here: Phase 4.3 sets up forwarding and NAT for IPv4 only, so a device with a working IPv6 route would be routing around the Pi rather than through it, and traffic that never crosses the gateway cannot be measured no matter what the capture layer supports. Expect to see no IPv6 traffic on this build. That is the intended outcome, not a fault.

`sudo nmtui` offers the same thing as an interactive menu if you prefer that to flags.

dhcpcd, older images only, appended to `/etc/dhcpcd.conf`:

```
interface eth1
static ip_address=192.168.50.1/24
```

Verify, whichever path you took:

```
ip addr show eth1     # expect inet 192.168.50.1/24
ip route              # default route MUST be on eth0, not eth1
```

On the NetworkManager path, confirm the settings resolved as intended rather than assuming the command that printed "successfully added" did what you meant. `nmcli` accepts unambiguous abbreviations, so a truncated value is accepted silently:

```
nmcli -g ipv4.method,ipv4.addresses,ipv4.never-default,ipv6.method   connection show ag-lan
```

Expected, in order:

```
manual
192.168.50.1/24
yes
disabled
```

Correct any that differ, for example:

```
sudo nmcli connection modify ag-lan ipv6.method disabled
sudo nmcli connection up ag-lan
```

> **NO-CARRIER here is normal**
> At this point `eth1` reports `NO-CARRIER` and its route is marked `linkdown`, because nothing is plugged into it yet. That clears when you cable the access point in Phase 7. The address and route are configured regardless.

4.2 DHCP and DNS

`dnsmasq` does two jobs. As a **DHCP server** it hands each device an address and tells it to use the Pi as its gateway, which is what routes traffic through the capture. As a **DNS forwarder** it resolves names on their behalf.

Create the config. Copy and paste this entire block, including the final EOF line. It writes the file in one go, so there is no editor to learn.

```
sudo tee /etc/dnsmasq.d/activitygraph.conf > /dev/null <<'EOF'
# Serve only on the LAN side, never on eth0 which faces the router.
interface=eth1
bind-interfaces

# Addresses for your devices. .1 to .49 stay free for fixed addresses
# such as the Pi (.1) and the access point (.2).
dhcp-range=192.168.50.50,192.168.50.200,12h

# Tell clients the Pi is their gateway and DNS server.
dhcp-option=option:router,192.168.50.1
dhcp-option=option:dns-server,192.168.50.1

domain=lan
log-queries
log-facility=/var/log/dnsmasq.log
EOF
```

Apply it, then check it actually started. A config error makes dnsmasq exit rather than run with defaults, so this is worth verifying:

```
sudo systemctl restart dnsmasq
systemctl is-active dnsmasq || sudo journalctl -u dnsmasq -n 20 --no-pager
```

Confirm it is listening on the LAN side only. Use ss's own port filter rather than grepping the text, which would also match unrelated ports such as 5353 and 53412:

```
sudo ss -lnup 'sport = :53'
```

Expect three lines, all belonging to dnsmasq:

Address	Meaning
192.168.50.1:53	The LAN side. This is the one that matters
127.0.0.1:53	Loopback, so the Pi resolves names for itself. Normal
[::1]:53	The IPv6 loopback equivalent. Also normal

Two things should **not** appear. `0.0.0.0:53` means bind-interfaces is not taking effect and the Pi is offering DNS to every network it touches. Your eth0 address, the 192.168.1.x one, means it is serving the upstream network, where a second DHCP and DNS server causes real problems.

Anything on port 5353 is `avahi-daemon` doing mDNS. It is unrelated to this setup and can be ignored.

> **If dnsmasq will not start: address already in use**
> Another process holds port 53, usually systemd-resolved. Run `sudo systemctl disable --now systemd-resolved` and then `sudo systemctl restart dnsmasq`.

Two settings are load-bearing. `bind-interfaces` keeps dnsmasq off eth0, so the Pi never offers DHCP upstream where a second DHCP server would cause real problems. The `option:router` line is what makes devices route through the Pi at all: without it they may receive the wrong gateway and bypass the capture entirely.

4.3 Forwarding and NAT

Two separate things are needed, and the Pi does neither by default.

- **Forwarding** lets the Pi pass packets between its two interfaces. Linux refuses by default, because a normal computer should not silently relay other machines' traffic.
- **NAT (masquerade)** rewrites outgoing packets to appear to come from the Pi. The upstream router knows nothing about your 192.168.50.x devices, so without this their replies have nowhere to return to.

Enable forwarding, permanently across reboots:

```
echo 'net.ipv4.ip_forward=1' | sudo tee /etc/sysctl.d/99-activitygraph.conf
sudo sysctl -p /etc/sysctl.d/99-activitygraph.conf

cat /proc/sys/net/ipv4/ip_forward     # expect 1
```

Back up any existing firewall rules before overwriting them:

```
[ -f /etc/nftables.conf ] && sudo cp /etc/nftables.conf /etc/nftables.conf.backup
```

Write the ruleset. Paste the whole block including the final EOF:

```
sudo tee /etc/nftables.conf > /dev/null <<'EOF'
#!/usr/sbin/nft -f
flush ruleset

table inet filter {
  # Allow traffic to pass between eth1 (devices) and eth0 (internet).
  chain forward { type filter hook forward priority 0; policy accept; }
}

table ip nat {
  chain postrouting {
    type nat hook postrouting priority 100; policy accept;
    # Rewrite outgoing packets to come from the Pi's WAN address.
    oifname "eth0" masquerade
  }
}
EOF
```

Apply it, make it load on boot, and verify it parsed:

```
sudo systemctl enable --now nftables
systemctl is-active nftables || sudo journalctl -u nftables -n 20 --no-pager
sudo nft list ruleset
```

The last command must show both the `forward` chain and the `masquerade` line. An empty ruleset means the file did not parse, even though `enable --now` reported success.

The printed ruleset will not match your file character for character, and that is expected:

You wrote	nft displays
priority 0	priority filter
priority 100	priority srcnat

Those are the symbolic names for those numeric priorities. Your comments are also absent, because nftables does not retain them in the loaded ruleset. Neither indicates a problem.

Nothing routes yet, because no devices are connected. You test this for real at the end of Phase 8.

Phase 5: Router 2 as an access point

> **Configure this router in isolation**
> Both routers ship on `192.168.1.1` with the same default SSID and admin password. Power router 1 OFF while you do this. With both live you get an address conflict, and a real chance of reconfiguring the wrong box without noticing.

Connect your laptop to router 2 and open `http://192.168.1.1`.

The easy path

Look for an **Operation Mode** setting offering **Access Point** alongside **Wireless Router**. Selecting Access Point does most of the following in one toggle and is far less error-prone.

The manual path

- **DHCP server: OFF.** The Pi is the DHCP server now. Two DHCP servers on one segment hand out conflicting leases and cause intermittent failures that are painful to diagnose.
- **LAN IP: static 192.168.50.2**, subnet 255.255.255.0. This must sit outside the 192.168.50.50 to .200 DHCP range configured earlier.
- **SSID: something distinct**, for example Home-AG. Do not reuse the old name.
- **Wi-Fi: ON.** This is the radio your devices will use.
- Its DSL and WAN ports stay unused. You are using only its switch and radio.

> **Beware the advice you will find online**
> Searching for how to reuse a router as an access point usually returns a double-NAT method: connect its **WAN** port and give it a different subnet. That works for extending Wi-Fi and is wrong here. It NATs every device before the traffic reaches the Pi, so the capture sees one address instead of your actual devices.

Where these settings live

The DHCP toggle is frequently **not** on a page of its own. On TP-Link gateways, and many others, both settings sit on the same page, with the checkbox below the IP fields:

```
Advanced > Network > LAN Settings

    IP Address      192.168.50.2
    Subnet Mask     255.255.255.0
```

```
DHCP Server     [ ] Enable      <- checkbox, below the IP fields
```

Make sure you are in the **Advanced** view rather than Basic, which usually hides LAN settings entirely. Changing both on one page is convenient: save once and the router drops off the network a single time rather than twice.

> **Saving disconnects you, and that is expected**
> The admin page is no longer at the address your browser is using. Reach it again at 192.168.50.2 once the Pi is serving DHCP in Phase 8, or immediately by giving your computer a temporary static address such as 192.168.50.10, mask 255.255.255.0, no gateway.

Phase 6: Router 1 becomes a modem

Power router 1 back on, open its admin page, and **turn off its Wi-Fi**, both the 2.4 GHz and 5 GHz radios. Leave everything else alone: it keeps the DSL connection and continues handing out addresses on `192.168.1.x`, which is how the Pi's `eth0` gets its address.

> **The single most common failure**
> Router 1 sits *above* the Pi. Any device that associates with its Wi-Fi reaches the internet without the Pi ever seeing it, and shows no traffic at all. If you also reused its SSID on router 2, devices may silently reattach to the wrong one.

Phase 7: Cabling

> **Do not change how router 1 reaches the internet**
> It is your existing working router, so whatever is plugged into it now is already correct. Leave that cable exactly as it is. The only change router 1 needs is its Wi-Fi turned off.

From	To
Your internet feed, unchanged	Router 1. A thin phone cable means the **DSL** port (FTTN or FTTB). An Ethernet cable from an NBN box or fibre NTD means the **WAN** port. Never a LAN port.
Router 1, any LAN port	Pi, built-in Ethernet (eth0)
Pi, USB adapter (eth1)	Router 2, any **LAN** port

Only the last two rows are new. The connectors are not interchangeable: a DSL port is RJ11, narrower with 6 pins, while WAN and LAN ports are RJ45, wider with 8 pins. An Ethernet plug will not seat properly in a DSL socket.

> **Use router 2's LAN port, never its WAN port**
> Cabling into the WAN port makes router 2 route and NAT the traffic before it reaches the Pi. That destroys per-device visibility in exactly the same way capturing on eth0 would.

Phase 8: Verify the plumbing [CHECKPOINT]

Connect your phone to router 2's Wi-Fi, then look at its IP address in the Wi-Fi settings.

Phone shows	Meaning
`192.168.50.x`, gateway `192.168.50.1`	Correct. Traffic is routing through the Pi.
`192.168.1.x`	Wrong. It is talking to router 1 and bypassing the Pi. Return to Phase 6 and confirm that radio is off.

On the Pi, confirm the lease was issued:

```
cat /var/lib/misc/dnsmasq.leases
```

Your phone should appear with its hostname. Confirm it also has working internet before continuing.

Phase 9: Run ActivityGraph

```
cd ~/activitygraph
sudo .venv/bin/python -m activitygraph.main \
  --serve --serve-source live -i eth1 \
  --host 0.0.0.0 --no-browser \
  --capture-mode light --flow-accounting --dhcp-leases \
  --accounts ~/activitygraph/accounts.db \
  --history-db ~/activitygraph/history.db
```

Flag	Why
`-i eth1`	The LAN side. **Never eth0.** ActivityGraph warns at startup if the interface you pass carries the default route.
`--dhcp-leases`	Learns a name for every device in the Pi's own lease table. Works with any router, needs no credentials. Names only: a lease records an address handed out and outlives the connection by hours, so a device appears once it sends traffic, wearing the name the lease taught.
`--active-within 15`	Limits the **sessions and the graph** to the last fifteen minutes, so a gateway that has run for weeks does not show an hour-old session as though it were live. It deliberately does not touch the time totals, the service rankings or the device list: those answer "where did today go", and cutting them to the window reported an hour of YouTube as zero the moment you stopped watching. A phone that has gone home stays listed, marked offline, with its time still counted.
`--enforce-blacklist`	Actually drops traffic from a group's devices to the sites it has blocked and the services whose daily limit it has spent. Refused without --accounts, since a block needs a group to apply to. See the section below before turning it on.
`--show-background`	Puts DNS, DHCP, clock sync, update checks and ad beacons back into the headline panels. Left out by default because none of it is something a person did, and DNS appears on every device. The raw flow list under Stats for Nerds always shows everything.
`--host 0.0.0.0`	Needed to reach the dashboard from another machine. It also publishes it to the whole LAN, which is why the flag below is on the same line.

Flag	Why
`--accounts`	Requires people to sign in, and shows each of them only the devices they have claimed. Without it the dashboard is unauthenticated and anyone who can reach the port can read every device's history.
`--enforce-blacklist`	What makes blocking and daily limits actually drop packets. **Without it the lists are recorded and nothing is blocked**, which looks exactly like the feature being broken. The dashboard says plainly when it is missing. See Phase 9.1.
`--capture-mode light`	Close to mandatory on a Pi. Every packet the capture filter admits is copied into Python and taken apart by scapy, which costs far more per packet than the kernel spends forwarding it. On a 110 Mbps line that is roughly 13,000 packets a second: the CPU saturates, forwarding starves, and measured throughput falls to about 15 Mbps. The same line measures about 100 Mbps in light mode, which inspects only what identifies devices, services and flow timing. Categories and time stay accurate; byte totals are a sample unless you add the flag below.
`--flow-accounting`	Makes the byte totals exact without giving up the throughput. Both problems come from assuming the only way to know about a packet is to look at it, and here that is not true: the Pi already does NAT, so the kernel's connection tracker holds an entry per flow with exact counters for both directions. Every packet is already counted, by code that sees them anyway, so reading that table every few seconds costs one pass over open connections rather than one pass over packets. It also fixes durations, because a flow whose counters moved is known to be alive rather than inferred from a sampled packet. Needs two things on Raspberry Pi OS, whose kernel is built without CONNTRACK_PROCFS so /proc/net/nf_conntrack does not exist even while tracking runs: `sudo apt install -y conntrack` to read the table over netlink, and `sudo sysctl -w net.netfilter.nf_conntrack_acct=1` because the counters are off by default. Without either, the service still runs and names the missing one at startup.
`--history-db`	Records activity over time, and stores hostnames learned from DNS so the service breakdown survives a reboot. A durable record of what every device did: treat it as private.

Open `http://192.168.50.1:8420` from any device on the network. Browse something distinctive on your phone, wait roughly 10 seconds for flows to close and classify, then check **Where Your Time Went** and **Time by Person**.

9.1 Blocking and daily limits

The Pi keeps itself out of the device list automatically: capturing an interface that does not carry the default route is what makes a machine a gateway rather than a laptop watching its own traffic, so no flag is needed. Pass `--include-self` to list it.

Each group can block sites outright, and can give itself a daily allowance for a service. Both are managed from the dashboard's **Daily Limits & Blocked Sites** panel: a person manages their own, and the admin sees and can edit every group's. Neither does anything to the network until `--enforce-blacklist` is added.

How enforcement is kept safe
Rules live in an nftables table of their own, called `activitygraph`. The filter and nat tables from Phase 4.3 are never touched, and the whole table is replaced in one atomic operation, so there is no moment where half a ruleset is live. If anything fails the table is removed rather than left partly applied: it fails open by design. Only forwarded traffic is dropped, so a group blocking half the internet keeps the dashboard, DNS and SSH, and with them the means to undo it.

A daily limit blocks only a service's front door. Over HTTPS the hostname is visible but the path is not, so youtube.com/watch and youtube.com/embed cannot be told apart; what can be told apart is which hosts each uses. An hour spent on YouTube blocks youtube.com and leaves googlevideo.com and youtube-nocookie.com reachable, so a video embedded in an article still plays and a map on a shop's contact page still loads. A service with no known front door is marked **not blockable** in the panel rather than appearing to work.

Blocking a domain covers its subdomains but not its siblings: blocking tiktok.com does not block tiktokcdn.com, which is a separate domain. Add it as well if you want the app fully unreachable. A device with a cached DNS answer can also reach the address until that cache expires, so a block is firm within about a minute rather than instantly.

Blocking is done by **naming the service**, not the address. The panel offers YouTube, TikTok, Instagram and the rest as one-click buttons, and typing youtube.com or pasting a YouTube link means the same thing. Either way the whole service goes: the website, the phone app, and the hosts the video actually comes from, which is why blocking a single domain used to leave the app working. A site that is not one of the known services is blocked by its domain, typed into the field beside the buttons. A daily limit deliberately does not do this, blocking the front door and leaving the rest reachable so an embedded video still plays. The two differ because the intent differs: a blacklist says none of this, a limit says enough of this.

9.1 Create the admin account first

Whoever registers first becomes the admin
The first account created can see every device on the network, and there is no other gate on it. Register yours immediately, before anyone else opens the dashboard.

Everyone else then registers their own account and starts out seeing nothing at all. They add a device by opening the dashboard **on that device** and pressing **Claim this device**, which asks them what to call it.

That name is the one everybody sees, prefixed with whose it is: Sam calling his phone "iPhone" appears as **Sam's iPhone**, and his group is **Sam's group**. Usernames are unique, so two people cannot produce the same label. This is asked for rather than guessed because what a device broadcasts about itself varies by make, by OS version and by whether it is awake, and phones in particular announce a bare model name. Its owner always knows what it is.

Claiming is checked against the connection's own source address, so a phone can only be added from the phone. That is what makes the dashboard safe to hand out: nobody can attach a device they are not holding, and nobody needs to be told anyone else's MAC address. A device belongs to one person at a time, so shared items such as a TV are usually left unclaimed, which leaves them visible to the admin only.

The admin cannot claim anything, deliberately. It already sees every device, so a claim would add nothing to its view, and because a device belongs to one group at a time it would take that device away from the person it belongs to, who would then have no way to claim it. Seeing everything and owning nothing is the whole role. Use a personal account as well if you want your own devices grouped.

The command above gives the account database an explicit path so that the service in Phase 10 uses the same one. Left bare, `--accounts` defaults to `~/.activitygraph/`, which resolves to root's home under `sudo`, and you would find yourself registering all over again once the service took over. The session key sits beside the database; deleting it signs everyone out. If you forget the admin password, delete both files and register again: that discards accounts and claims, but not captured history.

Phase 10: Grouping and permanence

10.1 Grouping devices without accounts

With `--accounts` on, people build their own groups by claiming devices and this step is unnecessary. The file-based alternative still exists for a gateway you administer alone, or if you would rather assign devices centrally than ask anyone to sign in.

Take the MAC addresses from the Devices panel and create `~/groups.json`:

```
{
  "Nathan": ["AA:BB:CC:00:00:01", "AA:BB:CC:00:00:02"],
  "Guest":  ["AA:BB:CC:00:00:03"]
}
```

Add `--device-groups ~/groups.json` to the command. Devices you do not list appear under Ungrouped, which is where shared items such as a TV naturally belong. The two mechanisms can run together: groups label the Time by Person panel, accounts decide who is allowed to see what.

10.2 Run it on boot

Set a variable to your project root first, so the paths are filled in for you rather than edited by hand:

```
AG=$(dirname "$(find ~ -name pyproject.toml -not -path '*/.venv/*' | head -1)")
echo "$AG"        # sanity check: this is your project root
```

Then paste the whole block, including the final EOF:

```
sudo tee /etc/systemd/system/activitygraph.service > /dev/null <<EOF
[Unit]
Description=ActivityGraph
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=root
WorkingDirectory=$AG
ExecStart=$AG/.venv/bin/python -m activitygraph.main --serve --serve-source live -i eth1 --host 0.0.0.0 -
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
EOF
```

Two deliberate choices there. The heredoc is unquoted so $AG is replaced with your real path as the file is written. And ExecStart is one long line rather than using backslash continuations, because a backslash inside an unquoted heredoc is consumed by the shell before systemd ever sees it, producing a mangled command.

Check the result, then enable it:

```
cat /etc/systemd/system/activitygraph.service
```

```
sudo systemctl enable --now activitygraph
systemctl is-active activitygraph || journalctl -u activitygraph -n 30
```

> **Systemd does not expand the ~ shortcut**
> Every path in the unit must be absolute. Confirm the file you just printed contains no $AG and no ~ characters. Note also that `pi` is no longer the default username on current Raspberry Pi OS images.

Troubleshooting

Symptom	Most likely cause
Phone gets a 192.168.1.x address	It associated with router 1's Wi-Fi. Turn that radio off.
Phone absent from dnsmasq.leases	Router 2's DHCP server is still on, or it is cabled by its WAN port instead of a LAN port.
Leases fine, but no traffic recorded	Capturing the wrong interface. It must be eth1.
Everything attributed to one IP address	Capturing eth0 post-NAT, or eth0 and eth1 swapped after a reboot. Check the udev rule from Phase 2.
Traffic appears but has no categories	Flows classify when they close. Allow about --idle-timeout seconds.
Dashboard unreachable	Confirm --host 0.0.0.0 and that you are browsing to 192.168.50.1, not the old 192.168.1.x address.
No internet anywhere	Check IP forwarding and the nftables masquerade rule from Phase 4.3.
Signed in, but every panel is empty	A non-admin account sees only the devices it has claimed. Open the dashboard on the device itself and claim it.
A lot of traffic filed as plain HTTPS	Services are identified from DNS, which the Pi only sees once devices start querying it. It fills in over the first few minutes, and --history-db carries it across reboots.
Everyone logged out after a reboot	The session key moved. It lives beside the path given to --accounts, so keep that path the same between the manual run and the service.
A phone shows as a bare model name	iOS sends a generic "iPhone" as its DHCP hostname, so the name its owner chose comes from Bonjour instead. That needs mDNS (UDP 5353) reaching the Pi, which --capture-mode light allows. Check Settings > General > About > Name, or override it with --device-aliases.
Time in categories looks too low	Without --flow-accounting, flows are kept alive by a sampled fraction of packets in light mode, so a quiet flow can be timed out while it is still open. Add the flag; if durations still look short, raise --idle-timeout.

Symptom	Most likely cause
Byte totals look too low	Expected in light mode without --flow-accounting. Check /api/health: flow_accounting.message says whether the kernel counters are being read, and names the fix if they are not.
No IPv6 traffic at all	Expected. The LAN is single-stack by design. See Phase 4.1.

Wi-Fi appears, but devices get no internet

The most common failure after the rebuild, and the one that looks worst while being the smallest fault. Establish which half is broken before changing anything, because the Pi is usually innocent.

Check the Pi first:

```
ip -br addr
ip route
ping -c 3 1.1.1.1
ping -c 3 google.com
```

You want eth0 holding an address from router 1, eth1 on 192.168.50.1/24, a default route out of eth0, and both pings succeeding. If all of that holds, the Pi is fine and nothing on it needs touching: the fault is between the Pi and your devices. Then check whether anything ever asked it for an address:

```
cat /var/lib/misc/dnsmasq.leases

ping -c 2 192.168.50.2

sudo tcpdump -i eth1 -n port 67 or port 68
```

> **An empty lease file with a healthy Pi means one thing**
> DHCP requests are not reaching the Pi at all. If 192.168.50.2 also does not answer, the access point is not on the Pi's LAN, and the cause is almost always that the cable from the Pi is in its WAN/Internet port rather than a numbered LAN port. In the WAN port it routes instead of bridging, so its Wi-Fi becomes a separate network and DHCP never crosses. With its own DHCP server switched off by then, clients associate and get no address at all, which presents as "connected, no internet". Run the tcpdump while connecting a phone: no output confirms it. Also rule out the access point's own DHCP still being enabled, and its Wi-Fi being a guest network, which isolates clients from the wired LAN just as effectively.

Things to know

- **The Pi becomes a single point of failure.** If it is off, the network is down. Restart=always covers crashes, not power loss or SD card corruption.
- **Packet inspection costs more than forwarding.** Measured on a Pi 4B on a 110 Mbps line: --capture-mode full gives about 15 Mbps, and --capture-mode light about 100 Mbps. Python cannot dissect 13,000 packets a second, and the pegged CPU starves the kernel forwarding path. Light mode avoids this by not looking at payload, and --flow-accounting then restores exact byte totals by reading the counters the kernel already keeps. Time in categories is unaffected.
- **Device-to-device traffic may bypass the Pi.** A phone casting to a TV, where both sit on router 2, is switched locally and never reaches the gateway. Only traffic crossing the Pi is visible.
- **You will lose SSH access briefly** during Phase 7, because your laptop is still on the old network. Move it to router 2's Wi-Fi and reach the Pi at 192.168.50.1.

- **The admin account sees everyone's activity.** Accounts stop household members reading each other's browsing; they hide nothing from whoever holds the admin login, or from anyone with shell access to the Pi. If you are setting this up for other people, tell them that rather than letting them assume otherwise.

Rollback

Nothing here is permanent. Turn router 1's Wi-Fi back on, reconnect your devices to it, and power the Pi off. Neither router was changed in a way that cannot be undone from its admin page.